

Email Header Analysis & Mail Tracking Detection Tool Using Python

Submitted By:

SANIVARAPU SUBBALAKSHMI

Course:

CYBER SECURITY

Institution:

SRKR ENGINEERING COLLEGE

Year:

2025

Abstract

Email remains one of the most widely used communication platforms and is also one of the most exploited attack vectors in cybersecurity. Attackers frequently use email spoofing, phishing campaigns, and malicious tracking mechanisms to deceive users. While email headers contain crucial routing and authentication information, interpreting raw header data requires technical expertise that most users lack.

This project presents a Python-based Email Header Analysis and Mail Tracking Detection Tool designed to simplify the process of analyzing raw email headers. The tool extracts the sender's originating IP address, checks authentication mechanisms such as SPF, DKIM, and DMARC, and identifies tracking or unsubscribe links embedded in emails.

The system processes raw header input and generates a structured, human-readable security report. The objective is to assist users in understanding whether an email is authenticated, potentially spoofed, or associated with tracking infrastructure.

This project demonstrates practical application of cybersecurity fundamentals using Python, string parsing, and regular expressions.

Introduction

Email communication is a fundamental component of modern digital infrastructure. Organizations rely on email for business operations, notifications, and marketing communication. However, email is also one of the most targeted attack surfaces in cybersecurity.

Attackers commonly use techniques such as:

- Email spoofing
- Phishing campaigns
- Domain impersonation
- Malicious tracking links

To mitigate such attacks, authentication mechanisms were introduced:

- **SPF (Sender Policy Framework)** – Verifies if the sending server is authorized.
- **DKIM (DomainKeys Identified Mail)** – Uses cryptographic signatures to validate message integrity.
- **DMARC (Domain-based Message Authentication, Reporting & Conformance)** – Builds policy enforcement on SPF and DKIM results.

Despite these protections, most users cannot interpret raw email headers, which contain routing paths, IP addresses, and authentication results in technical format.

This project focuses on building a Python-based tool that extracts essential security indicators from raw email headers and presents them in a simplified, structured format to assist in basic email forensic analysis.

Problem Statement

Email-based attacks continue to increase globally, with phishing and spoofing being among the most common techniques used by attackers. These attacks often rely on forged sender addresses and manipulated header information to appear legitimate.

Although email headers contain valuable information such as:

Sender IP address

Mail transfer path

SPF (Sender Policy Framework) validation

DKIM (DomainKeys Identified Mail) signatures

DMARC (Domain-based Message Authentication, Reporting & Conformance) results

most users are unable to interpret raw header data due to its technical format.

Additionally, marketing and phishing emails frequently include tracking links and embedded redirection URLs, which may compromise user privacy or redirect to malicious domains.

Currently, there is no simple tool available for non-expert users to quickly analyze raw email headers and understand authentication status.

Therefore, there is a need for an automated tool that simplifies email header analysis, extracts critical security information, and presents it in a readable and structured format.

PROPOSED SOLUTION & SYSTEM ARCHITECTURE

System Overview:

The proposed system is a Python-based email header analysis tool that performs the following steps:

1. Accepts raw email header input from the user.
2. Parses header content using string processing techniques.
3. Extracts the originating sender IP address.
4. Detects authentication results (SPF, DKIM, DMARC).
5. Identifies tracking and unsubscribe links.
6. Generates a formatted security analysis report.

Technologies Used:

1. Python Programming Language
2. Regular Expressions (Regex)
3. String Parsing Techniques
4. Command-Line Interface

Working Principle:

The tool scans the email header for specific keywords and patterns such as:

- “Received:” fields for IP extraction
- “spf=” for SPF result
- “dkim=” for DKIM result
- “dmarc=” for DMARC result

URLs for tracking detection

It then organizes the extracted information into a structured report.

IMPLEMENTATION

The Email Header Analysis Tool is implemented using Python due to its strong support for text processing, regular expressions, and simplicity in file handling. The system performs static analysis of raw email headers stored in a text file and generates a structured security report.

The implementation is modular and divided into functional components as described below.

6.1 Header Loading Module

The system begins by loading raw email header data from a text file (header.txt). The file is read in UTF-8 encoding to ensure compatibility with various email formats.

```
def load_headers(file_path):  
    with open(file_path, "r", encoding="utf-8") as f:  
        return f.read()
```

This function reads the entire header content into memory as a string, which is then passed to other modules for analysis.

6.2 Sender IP Extraction

Email headers contain multiple Received: fields because emails pass through several mail servers. To determine the originating sender IP, the tool:

1. Extracts all Received: lines using regular expressions.
2. Reverses the order of these entries.
3. Searches for the first valid IPv4 address pattern.

```
def extract_sender_ip(headers):  
    received_lines = re.findall(r"Received:.*", headers)  
    for line in reversed(received_lines):  
        ip_match = re.search(r"\b\d{1,3}(\.|\.|\d{1,3}){3}\b", line)
```

```
if ip_match:

    return ip_match.group()

return "Not found"
```

Reversing the list ensures that the earliest mail transfer agent (closer to the original sender) is analyzed first. The IPv4 address is detected using a regex pattern matching standard dotted decimal notation.

6.3 Authentication Results Extraction

Modern email security relies on three key authentication mechanisms:

- SPF (Sender Policy Framework)
- DKIM (DomainKeys Identified Mail)
- DMARC (Domain-based Message Authentication, Reporting & Conformance)

The tool scans the header for authentication results and classifies them as PASS, FAIL, or Not Found.

```
def extract_authentication_results(headers):

    results = {

        "SPF": "Not found",

        "DKIM": "Not found",

        "DMARC": "Not found"

    }

    lower_headers = headers.lower()

    if "spf=pass" in lower_headers or "received-spf: pass" in lower_headers:

        results["SPF"] = "PASS"

    elif "spf=fail" in lower_headers:

        results["SPF"] = "FAIL"

    if "dkim=pass" in lower_headers or "dkim-signature" in lower_headers:

        results["DKIM"] = "PASS (signature present)"
```

```

if "dmarc=pass" in lower_headers:

    results["DMARC"] = "PASS"

elif "dmarc=fail" in lower_headers:

    results["DMARC"] = "FAIL"

return results

```

The header content is converted to lowercase to ensure case-insensitive detection. The system identifies authentication results by searching for relevant keywords within the header text.

6.4 Tracking Link Detection

Marketing and automated emails often include tracking URLs for analytics and user behavior monitoring. The tool extracts all URLs and flags those containing tracking-related keywords.

```

def find_tracking_links(headers):

    urls = re.findall(r"https?:\/\/[^\s<>\"']+\"", headers)

    tracking_keywords = [

        "click", "track", "emailclick", "utm_",

        "moengage", "mailchimp", "hubspot", "sendgrid"

    ]

    flagged = []

    for url in urls:

        for key in tracking_keywords:

            if key in url.lower():

                flagged.append(url)

                break

    return flagged

```

URLs are extracted using regex and compared against a predefined list of known tracking-related keywords and marketing platforms. Any matching URLs are flagged in the final report.

6.5 Report Generation

The `main()` function integrates all modules and generates a structured console-based analysis report.

```
def main():  
  
    headers = load_headers("header.txt")  
  
    sender_ip = extract_sender_ip(headers)  
  
    auth_results = extract_authentication_results(headers)  
  
    tracking_links = find_tracking_links(headers)  
  
    print("\n Email Header Analysis Report")  
  
    print("-" * 45)  
  
    print(f"Sender IP Address: {sender_ip}")  
  
    print("\nAuthentication Results:")  
  
    for k, v in auth_results.items():  
  
        print(f" {k}: {v}")  
  
    print("\nTracking Links Found:")  
  
    if tracking_links:  
  
        for link in tracking_links:  
  
            print(f" {link}")  
  
    else:  
  
        print(" None detected")  
  
if __name__ == "__main__":  
  
    main()
```

The system outputs:

1. Sender IP Address
2. SPF, DKIM, DMARC status
3. List of flagged tracking URLs

The final output is formatted in a readable structure to simplify forensic interpretation

```

Email Header Analysis Report
-----
Sender IP Address: 193.35.16.213

Authentication Results:
SPF: PASS
DKIM: PASS (signature present)
DMARC: PASS

Tracking Links Found:
🔗 https://api-03.moengage.com/v1/emailunsubscribe?mqoe=Gx%2BTSNkmRwyQ7ddADUWNpbfvmQhsUeGa59In%2BUqZkQAXgZBVovaFDR88RBn4r1Tw9nxERENq60g-DIOs6vittPBxuE_tuUm%2Bmu86L
JhmjKfK2BR1_18gwXz1ui%2BzOCSEEX3mcmctzeJkSVSMg0RBHoEafA-2NrKWhGL8XsS11j_WRRg3szieAZyOtOhw5fd_hmumG3rdKQI10_1s25j60SZm9N2aY8Pg90680j3uN_X1be1KBj7NFu__cccD1_2MuvxJbpCVR
X7cuQkdSw1sV0o0Jfjc041B8VpPcu051Y-DeG5WmkSTJ206ke9aNMNakK%2BkwUxObAgZmrR61qcVpWvHztRlBgnc_66QcL8fhCytWWhCbLHq5G4a4jyCJTTeWY4-pwb7UfptU5FESEDEP63391Z1HgukoSuDQaM4G
Fsw4tg

```

- Missing authentication fields may indicate misconfiguration or potential spoofing.
- Presence of tracking links suggests marketing or automated email campaigns.
- The tool successfully converts complex header data into understandable results.

Testing confirms the tool performs accurate extraction of relevant header fields.

LIMITATIONS & FUTURE IMPROVEMENTS

Limitations:

- The tool does not perform live DNS validation for SPF records.
- It does not verify DKIM cryptographic signatures.
- Limited to static header analysis.
- Command-line interface only.

Future Enhancements:

- Integration with DNS lookup for real-time SPF validation.
- DKIM signature verification using cryptographic libraries.
- Development of a web-based interface.
- Phishing risk scoring system.
- Integration with threat intelligence APIs.

CONCLUSION

This project successfully demonstrates the development of an automated Email Header Analysis and Mail Tracking Detection Tool using Python. The system simplifies the interpretation of complex email header data and provides structured insights into sender IP address, authentication mechanisms, and tracking links.

The project strengthens understanding of email security fundamentals and highlights the importance of authentication protocols in preventing spoofing and phishing attacks.

This tool can serve as a foundational component for more advanced email forensic and cybersecurity systems.